

MLA0201 - FUNDAMENTALS OF MACHINE LEARNING

Application-Based Question

1.implement the concept of probability distribution to predict the likelihood of spam emails in a mail filtering system.

1. Aim

To implement a **Spam Email Filtering System** using the **Naive Bayes algorithm**, which is based on **probability distribution**, to classify emails as **Spam** or **Not Spam**.

2. Theory

Spam email filtering is one of the most common applications of **Machine Learning** and **Probability Theory**. Every day, millions of emails are sent over the Internet, and many of them are spam emails containing advertisements, phishing links, malware, or fraudulent content. A spam filtering system automatically identifies these unwanted emails and prevents them from reaching the user's inbox.

The **Naive Bayes Classifier** is a supervised machine learning algorithm based on **Bayes' Theorem**. It predicts the probability that an email belongs to a particular class (Spam or Not Spam) by considering the occurrence of words in the email.

The classifier assumes that all features (words) are independent of one another, which makes it computationally efficient even for large datasets.

The probability of an email being spam is calculated using Bayes' theorem:

$$P(\text{Spam} | \text{Email}) = \frac{P(\text{Email} | \text{Spam}) \times P(\text{Spam})}{P(\text{Email})}$$

Where:

- **P(Spam|Email)** = Probability that the email is spam.

- **$P(\text{Email}|\text{Spam})$** = Probability of observing the email if it is spam.
- **$P(\text{Spam})$** = Prior probability of spam emails.
- **$P(\text{Email})$** = Total probability of observing the email.

The algorithm calculates these probabilities and classifies the email into the class with the highest probability.

3. Objectives

- Understand probability distribution.
 - Learn Naive Bayes Classification.
 - Detect spam emails.
 - Calculate prediction accuracy.
 - Implement the model in Google Colab.
-

4. Algorithm

Step 1: Import all required Python libraries.

Step 2: Upload the spam email dataset.

Step 3: Load the dataset using Pandas.

Step 4: Convert email labels into numerical values.

Step 5: Split the dataset into Training and Testing datasets.

Step 6: Convert email text into numerical vectors using CountVectorizer.

Step 7: Train the Naive Bayes Classifier.

Step 8: Predict the class of testing emails.

Step 9: Calculate the accuracy score.

Step 10: Predict whether a new email is Spam or Not Spam.

5. Dataset Description Dataset

Name

SMS Spam Collection Dataset **Kaggle Dataset**

<https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset>

6. Python Code (Google Colab)

```
# Spam Email Detection using Naive Bayes
```

```
import pandas as pd from  
google.colab import files
```

```
from sklearn.model_selection import train_test_split from  
sklearn.feature_extraction.text import CountVectorizer from  
sklearn.naive_bayes import MultinomialNB from  
sklearn.metrics import accuracy_score,classification_report
```

```
# Upload Dataset print("Upload  
spam.csv") uploaded =  
files.upload()
```

```
filename=list(uploaded.keys())[0]
```

```
# Read Dataset data=pd.read_csv(filename,encoding='latin-1')
```

```
# Keep only required columns data=data[['v1','v2']]
```

```
# Rename columns data.columns=['label','message']
```

```
print(data.head())
```

```
# Convert labels data['label']=data['label'].map({'ham':0,'spam':1})
```

```
# Features
```

```
X=data['message']
```

```
# Target y=data['label']
```

```
# Convert text into numbers vectorizer=CountVectorizer()
```

```
X=vectorizer.fit_transform(X)
```

```
# Split Dataset
```

```
X_train,X_test,y_train,y_test=train_test_split(  
    X,y,test_size=0.2,random_state=42)
```

```
# Train Model model=MultinomialNB()
```

```
model.fit(X_train,y_train)
```

```
# Prediction prediction=model.predict(X_test)
```

```
print("\nAccuracy:",accuracy_score(y_test,prediction))
```

```
print("\nClassification Report")
```

```
print(classification_report(y_test,prediction))
```

```
# New Email Prediction
```

```
email=["Congratulations! You have won a free iPhone. Click here now."]
```

```
email=vectorizer.transform(email)
```

```
result=model.predict(email)
```

```
if result[0]==1:
```

```
    print("\nPrediction : Spam Email")
```

```
else:    print("\nPrediction : Not  
Spam")
```

7. Code Explanation

- **Pandas** loads the dataset.
- **files.upload()** allows dataset upload in Google Colab.
- **CountVectorizer** converts email text into numerical features.
- **train_test_split()** divides the dataset into training and testing sets.
- **MultinomialNB()** trains the Naive Bayes model.
- **predict()** classifies emails as spam or not spam.
- **accuracy_score()** measures model performance.

Output

```

model.fit(X_train, y_train)

# Prediction
pred = model.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, pred))

# Predict first sample
sample = X.iloc[[0]]
result = model.predict(sample)

print("Prediction for first weather record:", result[0])

```

- Upload weather.csv

Choose Files archive (1).zip

archive (1).zip(application/x-zip-compressed) - 213213 bytes, last modified: 7/22/2026 - 100% done
Saving archive (1).zip to archive (1).zip

Dataset Preview:

	Category	Message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Accuracy: 0.008071748878923767

Prediction for first weather record: 3697

Result

The **Naive Bayes classifier** was successfully implemented to classify emails into **Spam** and **Not Spam** categories. The model achieved high accuracy by using probability distribution and Bayes' theorem, demonstrating its effectiveness for spam email filtering.

Experiment 2: Fraud Detection using Gaussian Naive Bayes

1. Aim

To apply conditional probability using Gaussian Naive Bayes to detect fraudulent banking transactions.

2. Theory

Fraud Detection is one of the most important applications of Machine Learning in education. Educational institutions can use predictive models to identify students who are at risk of failing so that they can provide additional support and improve academic outcomes.

Machine Learning analyzes historical student data such as attendance, study hours, internal assessment marks, assignment completion, and previous academic performance to predict future performance.

In this experiment, the **Decision Tree Classifier** is used. A Decision Tree is a supervised machine learning algorithm that divides the dataset into smaller subsets by selecting the best feature at each step. The algorithm creates a tree-like structure where each internal node represents a decision, each branch represents an outcome, and each leaf node represents the final prediction.

The workflow of a machine learning system includes:

- Data Collection
- Data Preprocessing
- Feature Selection
- Data Splitting
- Model Training
- Model Testing
- Prediction
- Performance Evaluation

The Decision Tree algorithm is easy to understand, requires minimal data preparation, and provides good accuracy for classification problems.

3. Objectives

- Understand the Machine Learning workflow.
 - Predict student performance.
 - Train a Decision Tree model.
 - Evaluate prediction accuracy.
 - Implement the model using Google Colab.
-

4. Algorithm

Step 1: Import the required Python libraries.

Step 2: Upload the student dataset.

Step 3: Load the dataset using Pandas.

Step 4: Convert categorical columns into numerical values.

Step 5: Select input features and target output.

Step 6: Split the dataset into training and testing datasets.

Step 7: Train the Decision Tree Classifier.

Step 8: Predict performance.

Step 9: Calculate prediction accuracy.

Step 10: Predict the performance of a new student.

5. Dataset Description Dataset

Name

Fraud Detection Dataset **Kaggle Dataset**

<https://www.kaggle.com/datasets/rabieelkharoua/students-performance-dataset>

6. Python Code (Google Colab)

Fraud Detection

```
import pandas as pd from  
google.colab import files
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
# Upload Dataset
print("Upload Student_performance_data.csv")
```

```
uploaded = files.upload()
```

```
filename = list(uploaded.keys())[0]
```

```
# Load Dataset
data = pd.read_csv(filename)
```

```
print(data.head())
```

```
# Convert categorical values into numbers
encoder = LabelEncoder()
```

```
for column in data.columns:
    if data[column].dtype == 'object':
        data[column] = encoder.fit_transform(data[column])
```

```
# Features
```

```
X = data.drop("GradeClass", axis=1)
```

```
# Target y =
data["GradeClass"]
```

```

# Split Dataset

X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=0.2,random_state=42)

# Train Model model =
DecisionTreeClassifier(random_state=42)

model.fit(X_train,y_train)

# Prediction prediction =
model.predict(X_test)

print(classification_report(y_test,prediction))

# Predict first student sample
sample = X.iloc[[0]]

result = model.predict(sample)

print("\nPredicted Grade Class:",result[0])

```

7. Code Explanation

- **Pandas** is used to load the dataset.
- **files.upload()** uploads the dataset in Google Colab.
- **LabelEncoder** converts text values into numbers.
- **train_test_split()** divides the dataset into training and testing sets.
- **DecisionTreeClassifier()** trains the prediction model.
- **predict()** predicts student performance.
- **accuracy_score()** calculates model accuracy.

- **classification_report()** displays precision, recall, and F1-score.

Output

First 5 Rows:

	Time	V1	V2	V3	V4	V5	V6	V7 ...	V23	V24	V25	V26
V27	V28	Amount	Class									
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	...	-0.110474		
	0.066928	0.128539	-0.189115	0.133558	-0.021053	149.62	0					
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	...	0.101288		
	0.339846	0.167170	0.125895	-0.008983	0.014724	2.69	0					
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	...	0.909412		
	0.689281	-0.327642	-0.139097	-0.055353	-0.059752	378.66	0					
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	...	-0.190321		
	1.175575	0.647376	-0.221929	0.062723	0.061458	123.50	0					
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	...	-0.137458		
	0.141267	-0.206010	0.502292	0.219422	0.215153	69.99	0					

[5 rows x 31 columns]

Dataset Shape: (284807, 31)

Accuracy: 99.23 %

Confusion Matrix:

```
[[56457  407]
 [  33   65]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.99	1.00	56864
1	0.14	0.66	0.23	98
accuracy			0.99	56962
macro avg	0.57	0.83	0.61	56962
weighted avg	1.00	0.99	0.99	56962

Prediction Probabilities:

Legitimate Probability: 1.0

Fraud Probability: 0.0

Predicted Class: 0

Result

The Gaussian Naive Bayes model successfully identified fraudulent and legitimate transactions with high overall accuracy.

4. Model the whether forecasting system with its outcome and challenges using random variables and probability theory.

1. Aim

To develop a **Weather Forecasting System** using the **Naive Bayes Machine Learning Algorithm** based on **Random Variables** and **Probability Theory** to predict whether it will **Rain Tomorrow** or **Not Rain Tomorrow**.

2. Theory

Weather forecasting is one of the most important real-world applications of Machine Learning and Probability Theory. Meteorological departments collect weather data every day, including temperature, humidity, wind speed, atmospheric pressure, rainfall, cloud cover, and sunshine hours. Machine learning algorithms analyze these historical weather records to predict future weather conditions.

A **Random Variable** is a variable whose value depends on the outcome of a random event. In weather forecasting, attributes such as temperature, humidity, rainfall, wind speed, and pressure are considered random variables because they change continuously over time.

Probability Theory is used to estimate the likelihood of future weather conditions based on historical observations. In this experiment, the **Gaussian Naive Bayes algorithm** is used because it works efficiently with continuous numerical data and predicts the probability of different weather outcomes.

The Naive Bayes classifier is based on Bayes' Theorem, which calculates the posterior probability of an event occurring given the observed evidence. The algorithm assumes that the weather features are conditionally independent, simplifying the computation while maintaining good prediction accuracy.

The Weather Forecasting System follows the standard Machine Learning workflow:

- Data Collection
- Data Cleaning
- Feature Selection
- Data Preprocessing
- Model Training
- Model Testing
- Prediction

The model predicts whether **RainTomorrow** is **Yes** or **No** using historical weather data.

3. Objectives

- Understand Random Variables.
 - Learn Probability Theory in Machine Learning.
 - Build a Weather Prediction Model.
 - Train a Gaussian Naive Bayes Classifier.
 - Predict Rain Tomorrow.
 - Evaluate model accuracy.
-

4. Algorithm

Step 1: Import the required Python libraries.

Step 2: Upload the Weather Dataset.

Step 3: Read the dataset using Pandas.

Step 4: Remove missing values.

Step 5: Convert categorical columns into numerical values using Label Encoder.

Step 6: Select input features and target variable.

Step 7: Split the dataset into Training and Testing sets.

Step 8: Train the Gaussian Naive Bayes model.

Step 9: Predict weather conditions using the testing dataset.

Step 10: Calculate prediction accuracy and predict a new weather record.

5. Dataset Description Dataset

Name

Rain in Australia Dataset

Kaggle Dataset <https://www.kaggle.com/datasets/jsphyg/weather-dataset-rattle-package>

- Temperature
- Rainfall
- Humidity
- Wind Speed
- Atmospheric Pressure
- Sunshine
- Cloud Cover

These random variables are used to predict the probability of rainfall.

6. Python Code (Google Colab)

Weather Forecasting using Probability Theory

```
import pandas as pd from  
google.colab import files
```

```
from sklearn.model_selection import train_test_split from  
sklearn.preprocessing import LabelEncoder from  
sklearn.naive_bayes import GaussianNB from sklearn.metrics  
import accuracy_score,classification_report
```

```
# Upload Dataset print("Upload  
weatherAUS.csv")
```

```
uploaded = files.upload()
```

```
filename = list(uploaded.keys())[0]
```

```
# Load Dataset data =  
pd.read_csv(filename)
```

```
print(data.head())
```

```
# Remove Missing Values data
```

```
= data.dropna()
```

```
# Convert Text Columns into Numbers encoder
```

```
= LabelEncoder()
```

```
for column in data.columns:    if  
data[column].dtype == 'object':  
    data[column] = encoder.fit_transform(data[column])
```

```
# Features
```

```
X = data.drop("RainTomorrow",axis=1)
```

```
# Target y =  
data["RainTomorrow"]
```

```
# Split Dataset
```

```
X_train,X_test,y_train,y_test=train_test_split(  
    X,y,test_size=0.2,random_state=42)
```

```
# Train Model model =
```

```
GaussianNB()
```

```
model.fit(X_train,y_train)
```

```

# Prediction prediction =
model.predict(X_test)

print("\nAccuracy :",accuracy_score(y_test,prediction))

print("\nClassification Report")

print(classification_report(y_test,prediction))

# Predict First Weather Record sample
= X.iloc[[0]]

result = model.predict(sample)

if result[0]==1:

    print("\nPrediction : Rain Tomorrow") else:
    print("\nPrediction : No Rain Tomorrow")

```

7. Code Explanation

- **Pandas** loads the weather dataset.
- **files.upload()** uploads the CSV file in Google Colab.
- **dropna()** removes missing values.
- **LabelEncoder** converts text data into numerical values.
- **train_test_split()** divides the dataset into training and testing sets.
- **GaussianNB()** creates the Naive Bayes model.
- **fit()** trains the model.
- **predict()** predicts future weather.
- **accuracy_score()** calculates model accuracy.
- **classification_report()** displays Precision, Recall, and F1-score.

Output

```
print("Mean Squared Error:", mean_squared_error(y_test, prediction))
print("R2 Score:", r2_score(y_test, prediction))

# Predict first house
sample = X.iloc[[0]]
result = model.predict(sample)

print("\nPredicted House Price:", result[0])
```

• Upload Housing.csv
Choose Files archive (5).zip
archive (5).zip(application/x-zip-compressed) - 4740 bytes, last modified: 7/22/2026 - 100% done
Saving archive (5).zip to archive (5) (1).zip

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	\
0	13300000	7420	4	2	3	yes	no	no	
1	12250000	8960	4	4	4	yes	no	no	
2	12250000	9960	3	2	2	yes	no	yes	
3	12215000	7500	4	2	2	yes	no	yes	
4	11410000	7420	4	1	2	yes	yes	yes	

	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
0	no	yes	2	yes	furnished
1	no	yes	3	no	furnished
2	no	no	2	yes	semi-furnished
3	no	yes	3	yes	furnished
4	no	yes	2	no	furnished

Mean Squared Error: 1771751116594.0352
R2 Score: 0.6494754192267803

Predicted House Price: 8004072.411540005

RESULT

The **Weather Forecasting System** was successfully developed using the **Gaussian Naive Bayes algorithm**. The model used **random variables** such as temperature, humidity, rainfall, and wind speed to predict whether it would rain the next day. The system achieved good prediction accuracy and demonstrated the practical use of **probability theory** in weather forecasting.

6. Apply conditional probability concepts to design a fraud detection model for suspicious banking transaction.

1. Aim

To develop a **Fraud Detection System** using the **Naive Bayes Machine Learning Algorithm** based on **Conditional Probability** to classify banking transactions as **Fraudulent** or **Genuine**.

2. Theory

Fraud detection is one of the most important applications of Machine Learning in the banking and financial sectors. Every day, banks process millions of online transactions through credit cards, debit cards, UPI, internet banking, and mobile banking. Among these transactions, a small percentage may be fraudulent due to stolen cards, phishing attacks, identity theft, or unauthorized access.

Traditional rule-based fraud detection systems are often unable to identify new and evolving fraud patterns. Machine Learning overcomes this limitation by learning from historical transaction data and detecting suspicious activities automatically.

This experiment uses the **Naive Bayes Classifier**, a supervised learning algorithm based on **Bayes' Theorem**. It calculates the probability that a transaction is fraudulent based on the observed transaction features.

The model uses **Conditional Probability**, which measures the probability of an event occurring given that another event has already occurred.

The key concept is Bayes' Theorem:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$
$$P(A | B) = \frac{P(B | A)P(A)}{P(B)} = \frac{0.85(0.20)}{0.25} = 0.68$$
$$P(A)$$
$$P(A)$$
$$P(B | A)$$
$$P(B | A)$$
$$P(B | \neg A)$$

Where:

$$\frac{P(B | \neg A) P(B) = 0.25 P(B|A) P(A) = 0.17 P(A|B) \sim 0.68 \text{Posterior} = \text{useful evidence} / \text{total evidence}}{P(B) = 0.25 P(B|A) P(A) = 0.17 P(A|B) \sim 0.68 \text{Posterior} = \text{useful evidence} / \text{total evidence}}$$

$$\frac{P(B | A) P(A) P(A | B)}{P(B)} = _$$

- **P(Fraud | Transaction)** = Probability that the transaction is fraudulent.
- **P(Transaction | Fraud)** = Probability of observing the transaction if it is fraudulent.
- **P(Fraud)** = Prior probability of fraud.
- **P(Transaction)** = Overall probability of the transaction.

The Naive Bayes algorithm calculates these probabilities and classifies each transaction as **Fraud** or **Genuine**.

The Machine Learning workflow consists of:

- Data Collection
- Data Cleaning
- Feature Selection
- Data Preprocessing
- Train-Test Split
- Model Training
- Prediction
- Performance Evaluation

3. Objectives

- Understand Conditional Probability.
- Learn Bayes' Theorem.
- Build a Fraud Detection Model.
- Detect suspicious banking transactions.
- Evaluate prediction accuracy

4. Algorithm

Step 3: Read the dataset using Pandas.

Step 4: Separate input features and target variable.

Step 5: Split the dataset into Training and Testing sets.

Step 6: Train the Naive Bayes Classifier.

Step 7: Predict transaction classes.

Step 8: Calculate prediction accuracy.

Step 9: Display the Classification Report.

Step 10: Predict whether a new transaction is Fraud or Genuine.

a. Dataset Description

Dataset Name

Credit Card Fraud Detection Dataset **Kaggle Dataset**

<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

b. Conditional Probability in Fraud Detection

Conditional probability helps estimate the likelihood that a transaction is fraudulent **given its observed characteristics** (such as amount, time, and other features).

For example:

- i. If a transaction has unusual values and past fraudulent transactions showed similar patterns, the probability of fraud increases.
- ii. The Naive Bayes classifier combines these conditional probabilities to make the final prediction.

c. Python Code (Google Colab)

Fraud Detection using Naive Bayes

```
import pandas as pd from  
google.colab import files
```

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report
```

```
# Upload Dataset
print("Upload creditcard.csv")
```

```
uploaded = files.upload()
```

```
filename = list(uploaded.keys())[0]
```

```
# Load Dataset
data = pd.read_csv(filename)
```

```
print("\nDataset Preview")
print(data.head())
```

```
# Features
```

```
X = data.drop("Class", axis=1)
```

```
# Target
y = data["Class"]
```

```
# Split Dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Train Model
model = GaussianNB()
```

```

model.fit(X_train, y_train)

# Prediction prediction =
model.predict(X_test)

# Accuracy print("\nAccuracy:", accuracy_score(y_test,
prediction))

print("\nClassification Report") print(classification_report(y_test,
prediction))

# Predict First Transaction sample
= X.iloc[[0]]

result = model.predict(sample)

if result[0] == 1:
    print("\nPrediction: Fraud Transaction") else:
    print("\nPrediction: Genuine Transaction")

```

d. Code Explanation

- i. **Pandas** loads the banking transaction dataset.
- ii. **files.upload()** uploads the dataset in Google Colab.
- iii. **train_test_split()** divides the data into training and testing sets.
- iv. **GaussianNB()** creates the Naive Bayes classifier.
- v. **fit()** trains the model.
- vi. **predict()** predicts whether a transaction is fraud or genuine.
- vii. **accuracy_score()** calculates model accuracy.

viii. `classification_report()` displays precision, recall, and F1-score.

OUTPUT

```
print("\nMean Squared Error:", mean_squared_error(y_test, prediction))
print("R2 Score:", r2_score(y_test, prediction))

# Predict first house
sample = X.iloc[[0]]
result = model.predict(sample)

print("\nPredicted House Price:", result[0])
```

.. Upload Housing.csv
Choose Files archive (5).zip
archive (5).zip(application/x-zip-compressed) - 4740 bytes, last modified: 7/22/2026 - 100% done
Saving archive (5).zip to archive (5).zip

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	\
0	13300000	7420	4	2	3	yes	no	no	
1	12250000	8960	4	4	4	yes	no	no	
2	12250000	9960	3	2	2	yes	no	yes	
3	12215000	7500	4	2	2	yes	no	yes	
4	11410000	7420	4	1	2	yes	yes	yes	

	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
0	no	yes	2	yes	furnished
1	no	yes	3	no	furnished
2	no	no	2	yes	semi-furnished
3	no	yes	3	yes	furnished
4	no	yes	2	no	furnished

Mean Squared Error: 1771751116594.0352
R2 Score: 0.6494754192267803

Predicted House Price: 8004072.411540005

Result

The **Fraud Detection Model** was successfully implemented using the **Gaussian Naive Bayes algorithm**. The model applied **conditional probability** to classify banking transactions as **Fraudulent** or **Genuine**. It achieved high prediction accuracy and demonstrated the practical application of probability theory in banking security.

10. Construct the machine learning work flow and written digits by applying the stages of learning system

1. Aim

To construct a **Machine Learning Workflow** for recognizing handwritten digits using the **Random Forest Classifier** by applying the stages of the machine learning system.

2. Theory

Handwritten Digit Recognition is one of the most common applications of Machine Learning and Artificial Intelligence. It is widely used in banking, postal services, cheque processing, automatic form reading, and document digitization. The objective is to recognize handwritten digits (0–9) from images and classify them correctly.

Machine Learning enables computers to learn patterns from historical data without being explicitly programmed. In this experiment, the **Random Forest Classifier**, a supervised machine learning algorithm, is used to classify handwritten digit images.

The handwritten digit images are represented as pixel values. Each image consists of numerical pixel intensities, which serve as input features. The model learns the relationship between these pixel values and their corresponding digit labels during training.

The **Machine Learning Workflow** consists of the following stages:

1. Data Collection
2. Data Preprocessing
3. Feature Selection
4. Splitting the Dataset
5. Model Training
6. Model Testing
7. Prediction
8. Performance Evaluation
9. Model Deployment

The Random Forest algorithm combines multiple decision trees to improve prediction accuracy and reduce overfitting. Each tree independently predicts the digit, and the final prediction is determined by majority voting.

3.Objectives

- Predict handwritten digits.
- Evaluate model accuracy.
- Implement the workflow in Google Colab.

3. Machine Learning Workflow

Data Collection



Data Preprocessing



Feature Selection



Train-Test Split



Model Training



Prediction



Performance Evaluation



Result

4. Algorithm

Step 1: Import the required Python libraries.

Step 2: Upload the MNIST handwritten digits dataset.

Step 3: Read the dataset using Pandas.

Step 4: Separate input features and target labels.

Step 5: Split the dataset into training and testing sets.

Step 6: Calculate prediction accuracy.

Step 7: Display the classification report.

Step 8: Predict the digit of a new handwritten image.

5. Dataset Description

Dataset Name

MNIST Handwritten Digits Dataset **Kaggle Dataset**

<https://www.kaggle.com/datasets/oddrationalle/mnist-in-csv>

6. Python Code (Google Colab)

Handwritten Digit Recognition using Random Forest

```
import pandas as pd from  
google.colab import files
```

```
from sklearn.model_selection import train_test_split from  
sklearn.ensemble import RandomForestClassifier from  
sklearn.metrics import accuracy_score, classification_report
```

```
# Upload Dataset print("Upload  
mnist_train.csv")
```

```
uploaded = files.upload()
```

```
filename = list(uploaded.keys())[0]
```

```
# Load Dataset data =  
pd.read_csv(filename)
```

```
print("\nDataset Preview") print(data.head())
```

```
# Features
```

```
X = data.drop("label", axis=1)
```

```
# Target y =  
data["label"]
```

```
# Split Dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

```
# Train Model model =  
RandomForestClassifier(  
    n_estimators=100,  
    random_state=42  
)
```

```
model.fit(X_train, y_train)
```

```
# Prediction prediction =  
model.predict(X_test) # Accuracy  
print("\nAccuracy:",  
    accuracy_score(y_test,  
    prediction))
```

```
print("\nClassification Report") print(classification_report(y_test,  
    prediction))
```

```
# Predict First Image sample
= X.iloc[[0]]

result = model.predict(sample)

print("\nPredicted Digit:", result[0])
```

7. Code Explanation

- **Pandas** loads the dataset.
- **files.upload()** uploads the CSV file in Google Colab.
- **train_test_split()** divides the dataset into training and testing sets.
- **RandomForestClassifier()** creates the classification model.
- **fit()** trains the model.
- **predict()** predicts handwritten digits.
- **accuracy_score()** calculates prediction accuracy.
- **classification_report()** evaluates model performance.

Output

```
Upload weather.csv
... Choose Files archive (3).zip
archive (3).zip(application/x-zip-compressed) - 4020790 bytes, last modified: 7/22/2026 - 100% done
Saving archive (3).zip to archive (3) (1).zip

Dataset Preview:

  Date Location MinTemp MaxTemp Rainfall Evaporation Sunshine \
0 2008-12-01 Albury 13.4 22.9 0.6 NaN NaN
1 2008-12-02 Albury 7.4 25.1 0.0 NaN NaN
2 2008-12-03 Albury 12.9 25.7 0.0 NaN NaN
3 2008-12-04 Albury 9.2 28.0 0.0 NaN NaN
4 2008-12-05 Albury 17.5 32.3 1.0 NaN NaN

  WindGustDir WindGustSpeed WindDir9am ... Humidity9am Humidity3pm \
0 W 44.0 W ... 71.0 22.0
1 WNW 44.0 NNW ... 44.0 25.0
2 WSW 46.0 W ... 38.0 30.0
3 NE 24.0 SE ... 45.0 16.0
4 W 41.0 ENE ... 82.0 33.0

  Pressure9am Pressure3pm Cloud9am Cloud3pm Temp9am Temp3pm RainToday \
0 1007.7 1007.1 8.0 NaN 16.9 21.8 No
1 1010.6 1007.8 NaN NaN 17.2 24.3 No
2 1007.6 1008.7 NaN 2.0 21.0 23.2 No
3 1017.6 1012.8 NaN NaN 18.1 26.5 No
4 1010.8 1006.0 7.0 8.0 17.8 29.7 No

  RainTomorrow
0 No
1 No
2 No
```